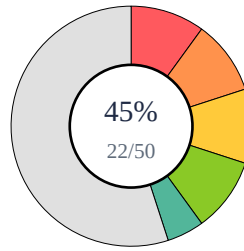


SCRIBIFY AI — STUDENT EVALUATION REPORT

Student ID: Dhaarun(345)



Total: 22/50
Percentage: 45.0%
Attempted: 3/5

Part A | Q1

Score: 0.0/10

Question: A university server needs to support multiple CPUs to run research workloads efficiently. Differentiate between symmetric and asymmetric multiprocessing. List three advantages and one disadvantage of multiprocessor systems.

Student Answer:

- **Correct:**
- **Wrong / Missing:** Answer not provided.
- **Improve:** Attempt the question with definition, key points, and example.
- **Ref:** Chunk 3; Chunk 1; Chunk 4

Part A | Q2

Score: 10.0/10

Question: A developer is working on an application that uses multiple threads to improve performance. During execution, the developer notices that the performance gain is negligible compared to a single-threaded version on a multi-core system. Explain why multithreading did not provide better performance in this case. Discuss two real-life programming examples where multithreading fails to improve performance.

Student Answer: Multi threading:: Multi threading improves performance by dividing a task into multiple threads that execute concurrently. Reasons for Negligible performance gain: Threads may be cpu-bound and compete for the same core resources, reducing benefits overhead of thread creation, Synchronization poorly designed programs may suffer from contention for shared resources Examples: file compression, GUI Applications.

- **Correct:** Your answer accurately explains why multithreading might not provide better performance, correctly identifying CPU-bound threads competing for resources, the overheads associated with thread management (creation, synchronization), and contention for shared resources in poorly designed programs. You also correctly provided two real-life programming examples: file compression and GUI applications, where multithreading can fail to improve performance.
- **Wrong / Missing:** Nothing significant was wrong or missing. Your answer covered all key aspects of the question comprehensively.
- **Improve:** Excellent work! Your answer is thorough and accurate. To make it even more explicit, when discussing overheads, you could specifically mention 'context switching' alongside thread creation and synchronization, but your current explanation is entirely sufficient.
- **Ref:** Textbook Context: 'Reasons for Negligible Performance Gain', 'Examples Where Multithreading Fails'

Part A | Q3

Score: 6.5/10

Question: A system administrator is asked to implement short-term, mid-term, and long-term scheduling policies. Describe each type with one example scenario where it is applied in real systems.

Student Answer: Scheduling policies: Scheduling is of three types. 1) Short-term scheduling (CPU scheduling) decides which process in the ready Queue gets the CPU next example: In a time-sharing system like a university server, Round Robin is used to give each students job equal CPU time 2) Mid-term Scheduling: Temporarily suspends or resumes processes to balance load and free resources. Example: Background research Jobs 3) Long term Scheduling (Job scheduling): Determines how many Jobs should be admitted into the system. Example: A batch system IPC Models Two fundamental IPC (Inter-process

Communication) models are 1) Message passing: processes communicate by sending and receiving messages Via OS primitives example: A chat application where the server and client exchange text messages Advantage: works well in distributed system where processes are on different Machines. 2) Shared memory: Multiple processes access a common memory region for communication example: A web browser where rendering and caching processes share memory for performance Advantage: Faster communication since no kernel involvement after setup.

- **Correct:** You accurately described short-term, mid-term, and long-term scheduling policies. Your example for short-term scheduling was also correct and comprehensive.
- **Wrong / Missing:** The example provided for mid-term scheduling, 'Background research Jobs', was too brief and did not fully articulate a scenario that demonstrates its application (e.g., *why* it would be suspended or resumed). Your example for long-term scheduling was incorrect and completely off-topic ('IPC Models'). Additionally, you included a section on 'IPC Models' which was not part of the question.
- **Improve:** When providing examples, ensure they describe a complete scenario that clearly illustrates how the scheduling policy works in practice. Always ensure your answer is focused solely on the question asked.
- **Ref:** Scheduling Policies

Part A | Q4

Score: 0.0/10

Question: A developer needs to implement Inter-Process Communication (IPC) between processes in a distributed environment. Explain the two fundamental IPC models with an example use-case for each.

Student Answer:

- **Correct:**
- **Wrong / Missing:** Answer not provided.
- **Improve:** Attempt the question with definition, key points, and example.
- **Ref:** Chunk 7; Chunk 5; Chunk 1

Part A | Q5

Score: 6.0/10

Question: An airline reservation system requires process creation and termination management for booking, cancellation, and payment services. Summarize the steps involved in process creation and process termination in an operating system. Provide a diagram illustrating these steps.

Student Answer: Airline reservation System. Each service like booking, cancellation, or payment runs as a separate process in the system. Process creation!: 1) A request comes to start a new process (e.g. booking) 2) os gives memory and resource to it 3) os creates a process control block to store process details 4) The program is loaded into memory. 5) The process moves to the Ready state and waits to run. Process Termination!: 1) The process finishes its work or is cancelled 2) os frees all resources used by it 3) The parent process is informed 4) os deletes the PCB

- **Correct:** You correctly identified several key steps for process creation, including the allocation of memory/resources, creation of the Process Control Block (PCB), and moving the process to the Ready state. For process termination, you correctly mentioned the process finishing/being cancelled, deallocating resources, and deleting the PCB.
- **Wrong / Missing:** For process creation, you missed the assignment of a unique Process ID (PID) and the explicit step of the scheduler selecting the process for execution. For process termination, you missed the handling of child processes. Additionally, the question explicitly asked for a diagram illustrating these steps, which was not provided.
- **Improve:** Review all specified steps for both creation and termination in the textbook, particularly PID assignment, scheduler selection, and child process handling. Ensure you include any requested diagrams to fully answer the question.
- **Ref:** Q20. Process creation and termination